UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

CURSO DE CIÊNCIA DA COMPUTAÇÃO

ANDREI POCHMANN KOENICH

MATRÍCULA 00308680

TURMA B

RELATÓRIO - TRABALHO PRÁTICO 3

Disciplina: Fundamentos de Processamento de Imagens

Professor: Manuel Menezes de Oliveira Neto

Porto Alegre, março de 2022.

INTRODUÇÃO

O relatório a seguir tem por objetivo demonstrar as operações envolvendo processamento de imagens que foram implementadas no Trabalho Prático 3 da disciplina. Esse trabalho envolve realizar operações sobre as imagens capturadas por uma câmera (processamento de vídeo).

O desenvolvimento do trabalho envolveu a utilização de linguagem C++, com Visual Studio 2022 como ambiente de programação. Para as funções de processamento de imagens, utilizou-se a biblioteca OpenCV, enquanto que para as funções envolvendo a criação da interface, utilizou-se a biblioteca CVUI.

As operações de processamento de vídeo, a partir do uso da câmera conectada ao computador, consistem em:

- Captura e exibição do vídeo da câmera em tempo real, após a inicialização do programa;

- Aplicação de borradura ao vídeo com filtragem Gaussiana;

- Detecção de arestas;

- Estimativa de gradiente do vídeo;

- Ajuste de brilho e contraste;

- Obtenção de negativo;

- Conversão de cores para tons de cinza;

- Redimensionamento do tamanho do vídeo para a metade do número de pixels, em cada uma de suas dimensões;

- Rotação do vídeo em noventa graus;

- Espelhamento horizontal e vertical;

- Gravação de um vídeo a partir das imagens capturadas pela câmera.

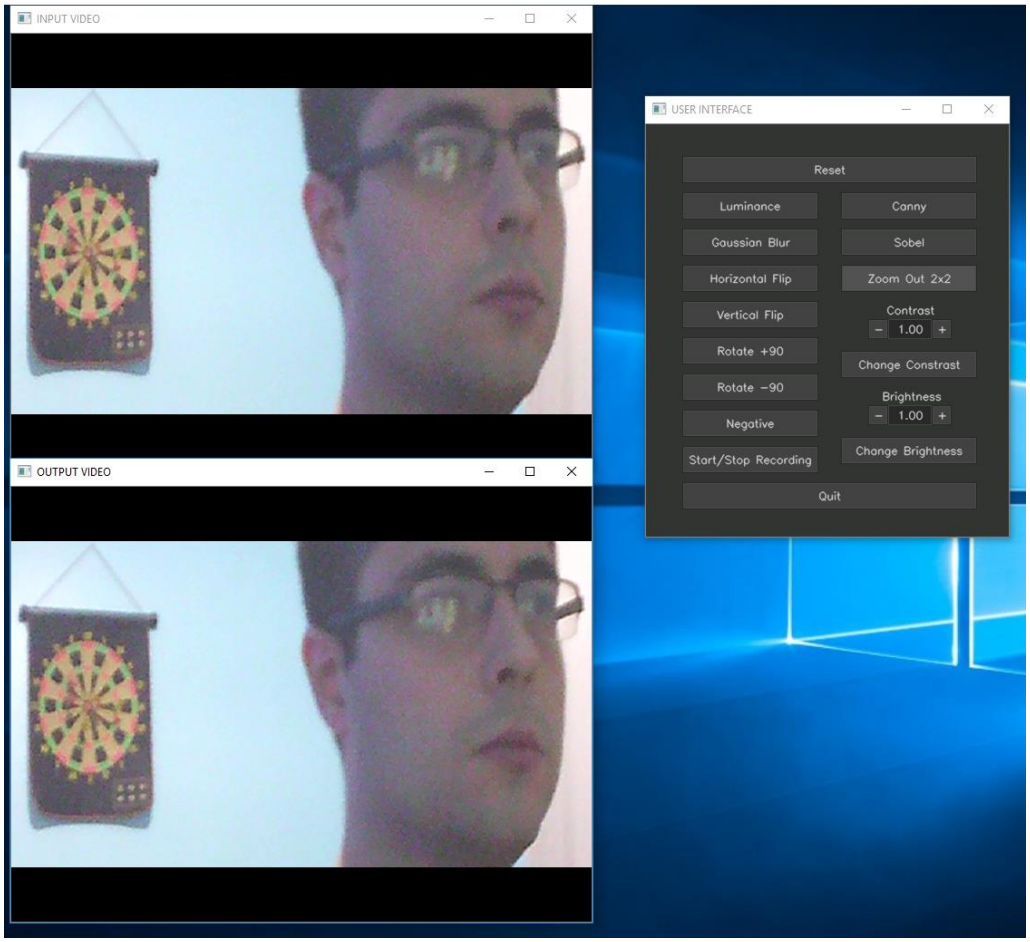
1) Captura e exibição do vídeo da câmera em tempo real, após a inicialização do programa

A captura e exibição do vídeo da câmera em tempo real utilizou como base a função fornecida na especificação do trabalho prático, mostrada abaixo.

Apêndice A: Programa básico para captura e exibição de video em tempo real

```
#include <opencv2/opencv.hpp>
using namespace cv;

int main(int argc, char** argv)
{
    int camera = 0;
    VideoCapture cap;
    // open the default camera, use something different from 0 otherwise;
    // Check VideoCapture documentation.
    if(!cap.open(camera))
        return 0;
    for(;;)
    {
          Mat frame;
          cap >> frame;
          if( frame.empty() ) break; // end of video stream
          imshow("This is you, smile! :)", frame);
          if( waitKey(1) == 27 ) break; // stop capturing by pressing ESC
    }
    cap.release();  // release the VideoCapture object
    return 0;
}
```

Uma vez que o programa é inicializado, mostra-se uma janela exibindo os frames capturados pela câmera e, abaixo, outra janela equivalente. As operações escolhidas pelo usuário com o uso da interface (a qual é exibida em uma terceira janela ao lado das duas primeiras) são aplicadas sobre a janela localizada na parte de baixo. A imagem a seguir demonstra essa etapa inicial do programa.



A interface permite que o usuário possa desfazer todas as modificações feitas na captura em tempo real pela câmera, clicando no botão “Reset” da interface.

Abaixo, seguem os trechos do programa que possibilitam essa implementação, que foram adaptados a partir do programa básico já mostrado.

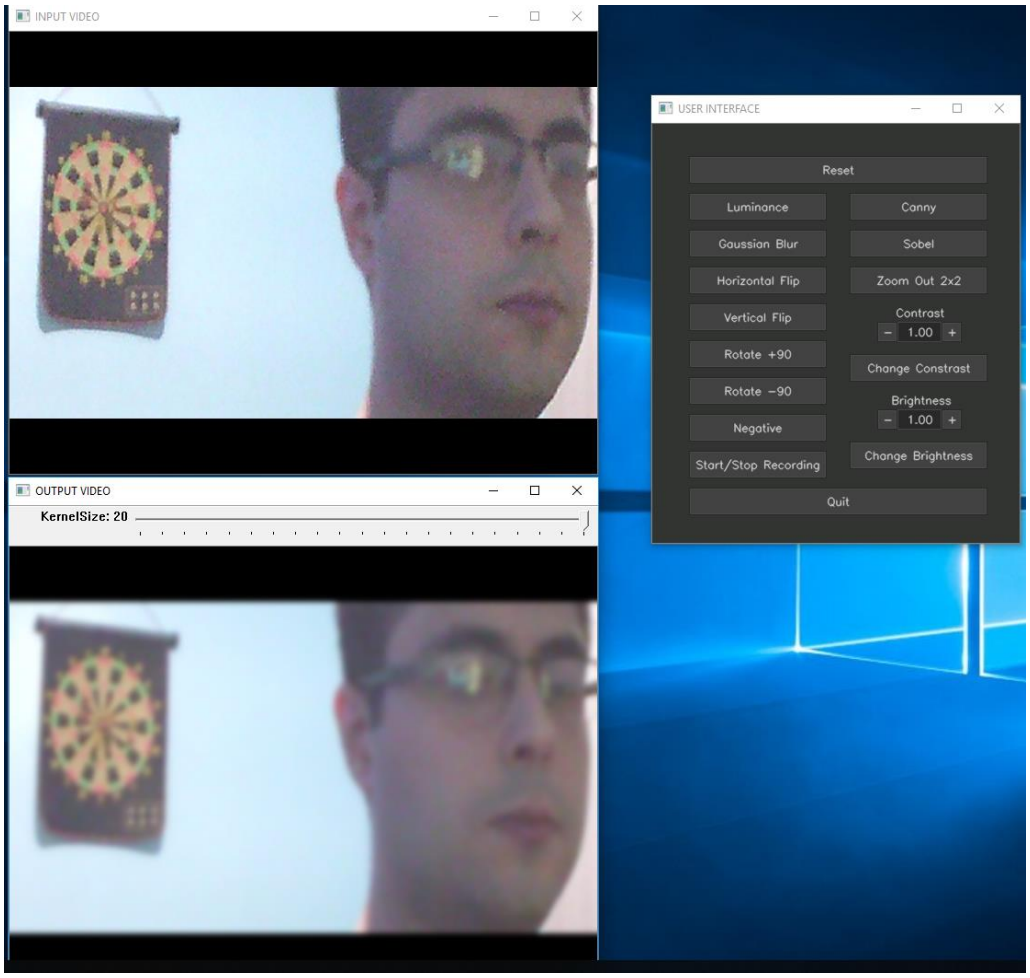
```
void inicia_programa()
{
    Mat frame_interface = Mat(ALTURAINTERFACE, LARGURAINTERFACE, CV_8UC3); /* Declaracao do frame que corresponde a janela de interface. */

    namedWindow(NOMEINTERFACE);
    moveWindow(NOMEINTERFACE, X_INTERFACE, Y_INTERFACE); /* Posiciona a janela da interface em local adequado na tela. */
    cvui::init(NOMEINTERFACE);

    int cameraIn = 0;

    VideoCapture cap;

    if (!cap.open(cameraIn)) /* Abre a camera padrao conectada. */
        return;

    int frame_largura = cap.get(cv::CAP_PROP_FRAME_WIDTH);
    int frame_altura = cap.get(cv::CAP_PROP_FRAME_HEIGHT);
    VideoWriter video(NOMEVIDEOSAIDA, cv::VideoWriter::fourcc('M', 'J', 'P', 'G'), 10, Size(frame_largura, frame_altura));

    double contrast = 1;
    double brightness = 1;

    int comandos[TOTALCOMANDOS] = { 0 }; /* Vetor destinado a conter o numero de vezes que cada um dos 12 comandos foi executado. */
    int tamanho_kernel = 3;

    Mat inCamera; /* Variaveis para conter o frame do video detectado pela camera. */
    Mat outCamera;
    Mat auxCamera;

    while (true) /* Iteração para processamento de cada frame capturado pela camera. */
    {
        cap >> inCamera;
        cap >> outCamera;

        if (inCamera.empty())
            break; /* Fim do streaming da camera. */

        if (outCamera.empty())
            break; /* Fim do streaming da camera. */
```

Em seguida, visualizamos os trechos de código no final da iteração que processa cada frame da câmera.

```
        namedWindow(NOMEJANELACAMERAIN, WINDOW_AUTOSIZE);
        moveWindow(NOMEJANELACAMERAIN, X_CAMERAIN, Y_CAMERAIN); /* Posiciona a janela da interface em local adequado na tela. */
        imshow(NOMEJANELACAMERAIN, inCamera); /* Mostra o frame da iteracao da vez. */

        namedWindow(NOMEJANELACAMERAOUT, WINDOW_AUTOSIZE);
        moveWindow(NOMEJANELACAMERAOUT, X_CAMERAOUT, Y_CAMERAOUT); /* Posiciona a janela da interface em local adequado na tela. */
        imshow(NOMEJANELACAMERAOUT, outCamera); /* Mostra o frame da iteracao da vez, apos alguma modificacao pelas funcoes. */

        cvui::update(NOMEINTERFACE);
        cvui::imshow(NOMEINTERFACE, frame_interface);

        if (waitKey(1) == ESC)
            break; /* Encerra a captura da camera quando o usuario pressiona ESC. */
    }

    video.release(); /* Libera o objeto video, apos o fim da iteracao. */
    cap.release(); /* Libera o objeto VideoCapture, apos o fim da iteracao. */
}
```

2) Utilização de *GaussianBlur* para aplicar borramento ao vídeo capturado pela câmera

Para aplicar o efeito de borramento, o usuário deve clicar no botão “Gaussian Blur” na interface. A partir disso, o programa irá exibir um *trackbar* na janela inferior, permitindo que o usuário ajuste o tamanho do kernel Gaussiano utilizado na operação, conforme demonstrado nas imagens abaixo.

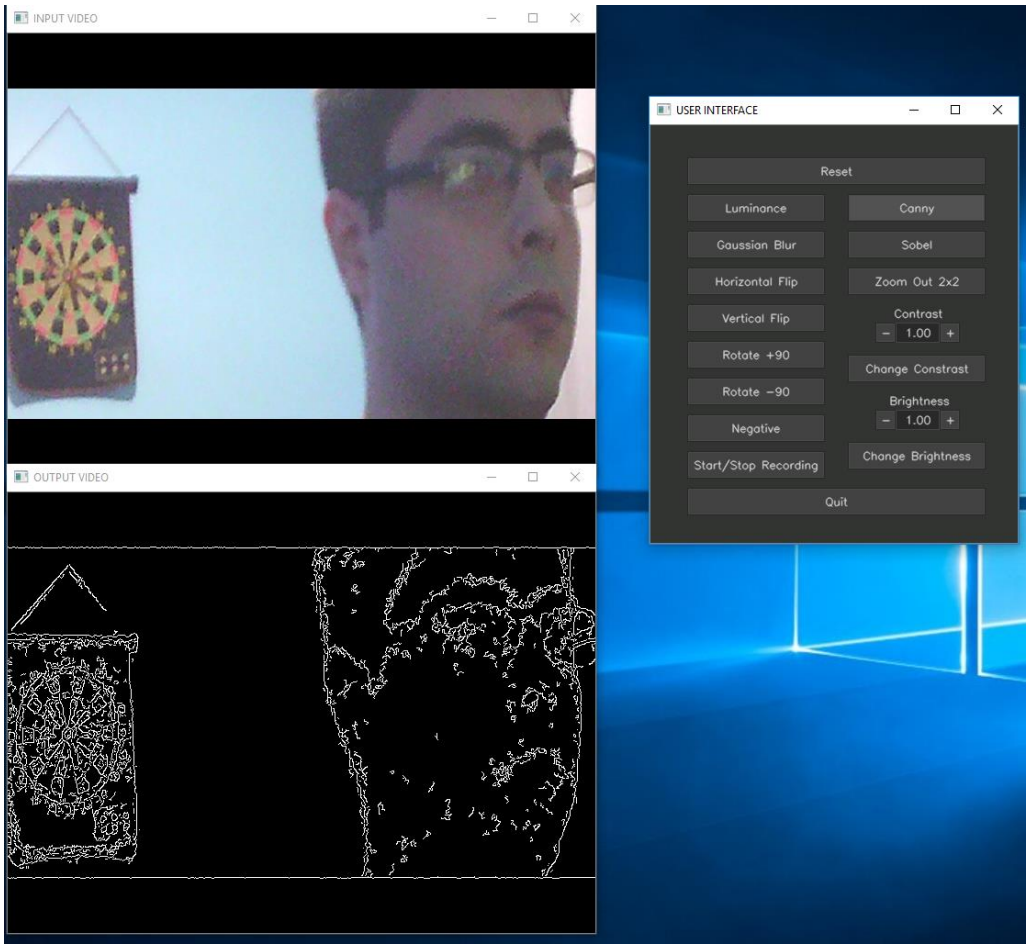


Uma vez que o usuário pressiona o botão “Reset” na interface após ter pressionado o botão “Guassian Blur”, além de desfazer a operação de borramento, o programa também omite o *trackbar* que foi gerado. Abaixo, visualizamos os trechos de código que permitem a implementação dessa operação. Utilizou-se os comandos **createTrackbar()** e **GaussianBlur()** da biblioteca OpenCV.

```
if (comandos[OP_LUMINANCIA] != 0) /* Verifica se o vetor de comandos contem a operacao de luminancia escolhida. */
{
    auxCamera = outCamera.clone();
    cvtColor(auxCamera, outCamera, COLOR_BGR2GRAY);
    cvtColor(outCamera, outCamera, COLOR_GRAY2BGR);
}

if (cvui::button(frame_interface, 40, 115, LARGURABOTAO, ALTURABOTAO, "Gaussian Blur")) /* Aplica borradura com o filtro Gaussiano. */
{
    createTrackbar("KernelSize", NOMEJANELACAMERAOUT, &tamanho_kernel, LIMITETRACKBAR);

    if (comandos[OP_GAUSSIANBLUR] == 0) /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */
        comandos[OP_GAUSSIANBLUR] = 1;
}

if (comandos[OP_GAUSSIANBLUR] != 0) /* Verifica se o vetor de comandos contem a operacao de Gaussian Blur escolhida. */
    GaussianBlur(outCamera, outCamera, Size(2 * tamanho_kernel + 1, 2 * tamanho_kernel + 1), 3, 0); /* Aplica operacao de borradura, garantindo que o kernel sempre possua dimensoes impares. */
```

3) Utilização do comando *Canny* para detectar arestas no vídeo

Para aplicar o efeito de detecção de arestas, o usuário deve clicar no botão “Canny” na interface. A imagem abaixo demonstra a utilização dessa funcionalidade no programa.

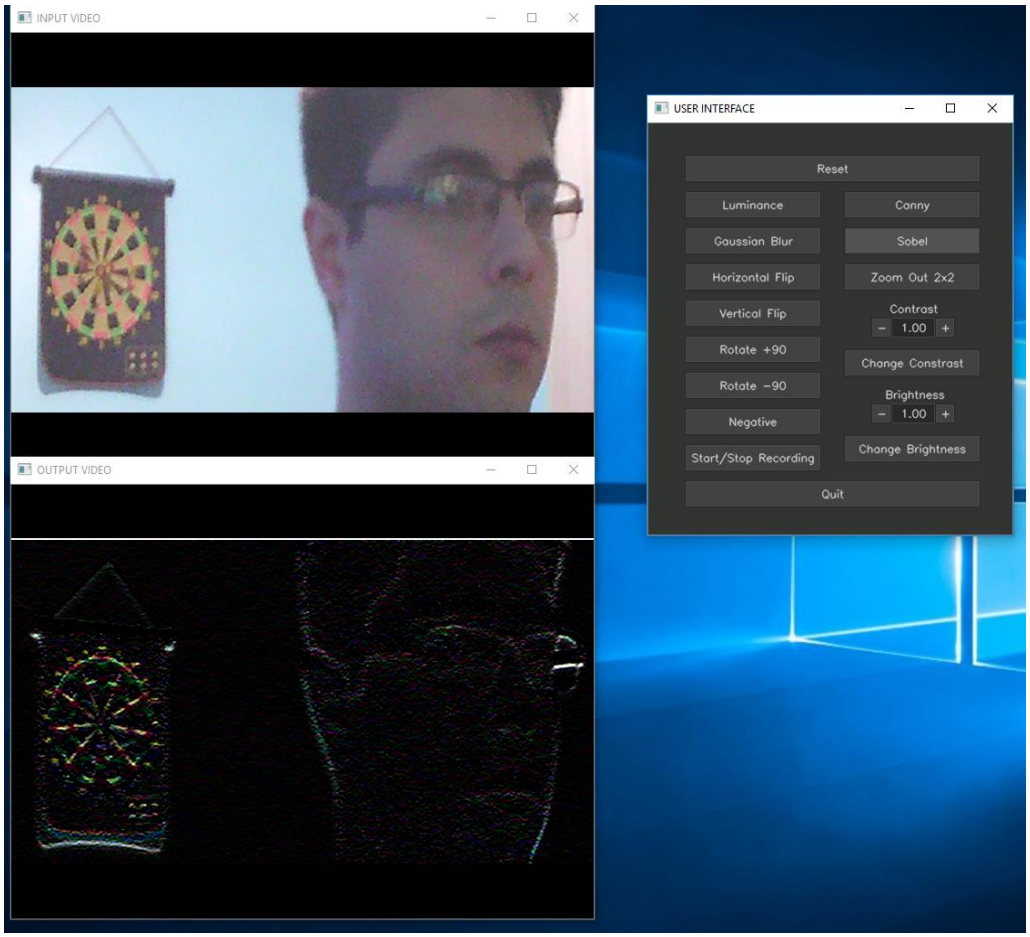


Abaixo, visualizamos os trechos do código que permitem essa implementação envolvendo o *Canny*. Utilizou-se o comando **Canny()** da biblioteca OpenCV.

```
if (cvui::button(frame_interface, 215, 75, LARGURABOTAO, ALTURABOTAO, "Canny")) /* Utiliza o comando Canny para detectar arestas no video. */
    comandos[OP_CANNY]++; /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */

if (comandos[OP_CANNY] != 0) /* Verifica se o vetor de comandos contem a operacao de Canny escolhida. */
{
    for (int i = 0; i < comandos[OP_CANNY]; i++) /* Realiza a operacao de Canny o numero de vezes necessario. */
    {
        auxCamera = outCamera.clone();
        Canny(auxCamera, outCamera, 50, 100);
        cvtColor(outCamera, outCamera, COLOR_GRAY2BGR);
    }
}
```

4) Utilização do comando *Sobel* para obter uma estimativa do gradiente do vídeo

Para obter uma estimativa do gradiente do vídeo, o usuário deve clicar no botão “Sobel” na interface. A imagem abaixo demonstra a utilização dessa funcionalidade no programa.



Abaixo, visualizamos os trechos do código que permitem essa implementação envolvendo o *Sobel*. Utilizou-se o comando **Sobel()** da biblioteca OpenCV.

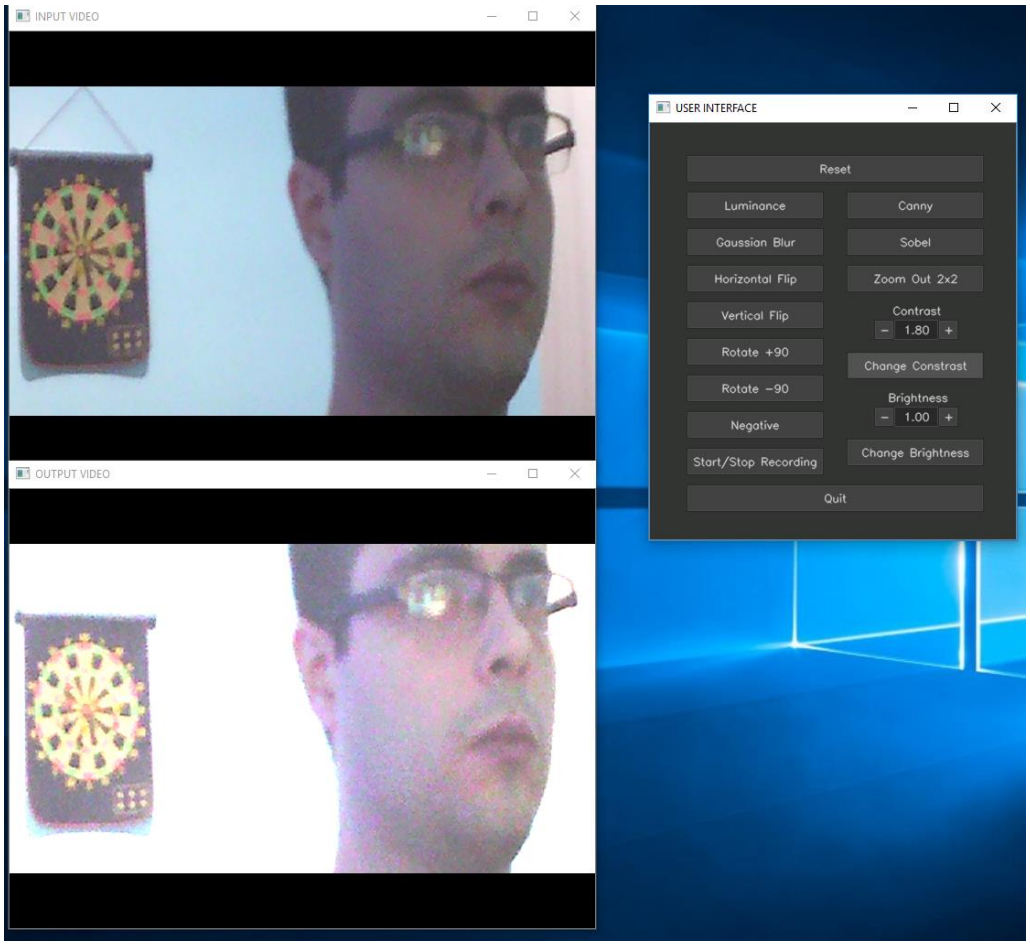
```
if (cvui::button(frame_interface, 215, 115, LARGURABOTAO, ALTURABOTAO, "Sobel")) /* Utiliza o comando Sobel para obter uma estimativa do gradiente do video. */
    comandos[OP_SOBEL]++; /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */

if (comandos[OP_SOBEL] != 0) /* Verifica se o vetor de comandos contem a operacao de Sobel escolhida. */
{
    for (int i = 0; i < comandos[OP_SOBEL]; i++) /* Realiza a operacao de Sobel o numero de vezes necessario. */
        Sobel(outCamera, outCamera, CV_8UC1, 0, 1, 3, 1, 1, BORDER_DEFAULT);
}
```

5) Ajuste de contraste no vídeo

Para alterar o valor do contraste dos frames obtidos pela câmera, inicialmente o usuário deve clicar nos botões ‘’+’’ e ‘’-‘ presentes na interface, abaixo do texto “Contrast”, para escolher o valor do escalar que será multiplicado por cada pixel do frame.

O valor inicial do escalar na interface é 1,00, e a cada clique do usuário em “+” ou “-“ o valor do escalar a ser multiplicado pelos pixels do frame é incrementado ou decrementado em 0,10. Uma vez alterado o valor, para a alteração efetiva do brilho e exibição da imagem alterada, o usuário deve clicar em “Change Contrast”. O resultado da operação de ajuste de contraste em cada pixel da imagem garantidamente estará na faixa [0, 255], uma vez que o programa realiza o ajuste necessário para manter os valores dentro desse intervalo (ajustando o valor para 255 quando necessário). Caso o usuário escolha valores menores ou iguais a zero ou maiores do que 255, o programa assume que cada pixel será multiplicado por 0,10 ou 255, respectivamente.

A imagem abaixo demonstra a utilização dessa funcionalidade no programa. Nesse teste, utilizou-se um escalar igual a 1,80 na operação.



Abaixo, visualizamos os trechos do código que permitem essa implementação de mudança de contraste. Utilizou-se o comando **convertTo()** da biblioteca OpenCV.

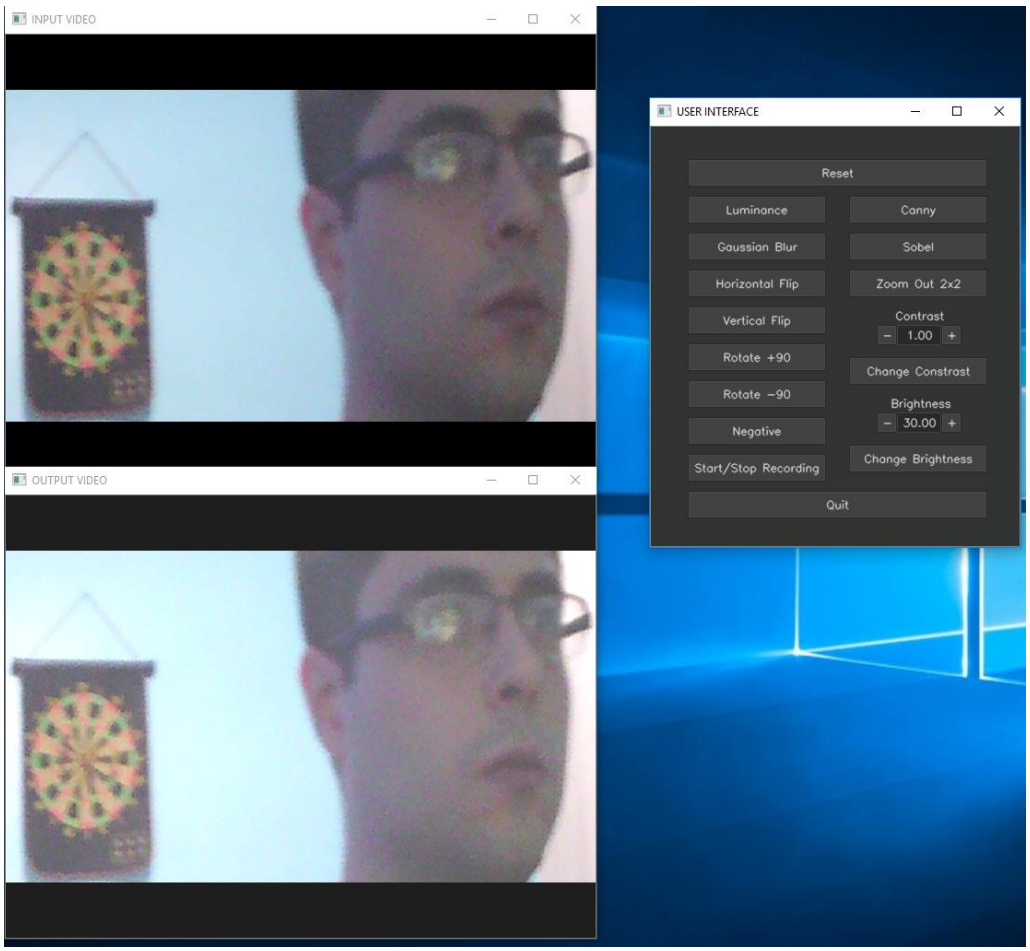
```
if (cvui::button(frame_interface, 215, 250, LARGURABOTAO, ALTURABOTAO, "Change Contrast")) /* Altera o contraste do video. */
    comandos[OP_CONTRASTE] = 1; /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */

if (comandos[OP_CONTRASTE] != 0) /* Verifica se o vetor de comandos contem a operacao de mudanca de contraste escolhida. */
    outCamera.convertTo(outCamera, -1, contrast, 0); /* Faz a mudanca de contraste. */
```

6) Ajuste de brilho no vídeo

Para alterar o valor do brilho dos frames obtidos pela câmera, inicialmente o usuário deve clicar nos botões ‘’+’’ e ‘’-‘ presentes na interface, abaixo do texto “Brightness”, para escolher o valor do escalar que será somado a cada pixel do frame.

O valor inicial do escalar na interface é 1,00, e a cada clique do usuário em “+” ou “-“ o valor do escalar a ser somado com os pixels da imagem é incrementado ou decrementado em 0,50. Uma vez alterado o valor, para a alteração efetiva do brilho e exibição da imagem alterada, o usuário deve clicar em “Change Brightness”. O resultado da operação de ajuste de brilho em cada frame garantidamente estará na faixa [0, 255], uma vez que o programa realiza o ajuste necessário para manter os valores dentro desse intervalo (ajustando o valor para 0 ou 255 quando necessário). Caso o usuário escolha valores menores do que -255 ou maiores do que 255, o programa assume que cada pixel será somado com -255 ou 255, respectivamente.

A imagem abaixo demonstra a utilização dessa funcionalidade no programa. Nesse teste, utilizou-se um escalar igual a 30,00 na operação. Foram feitas manipulações da imagem em baixo nível, de forma similar ao que foi realizado no Trabalho Prático 2 da disciplina.

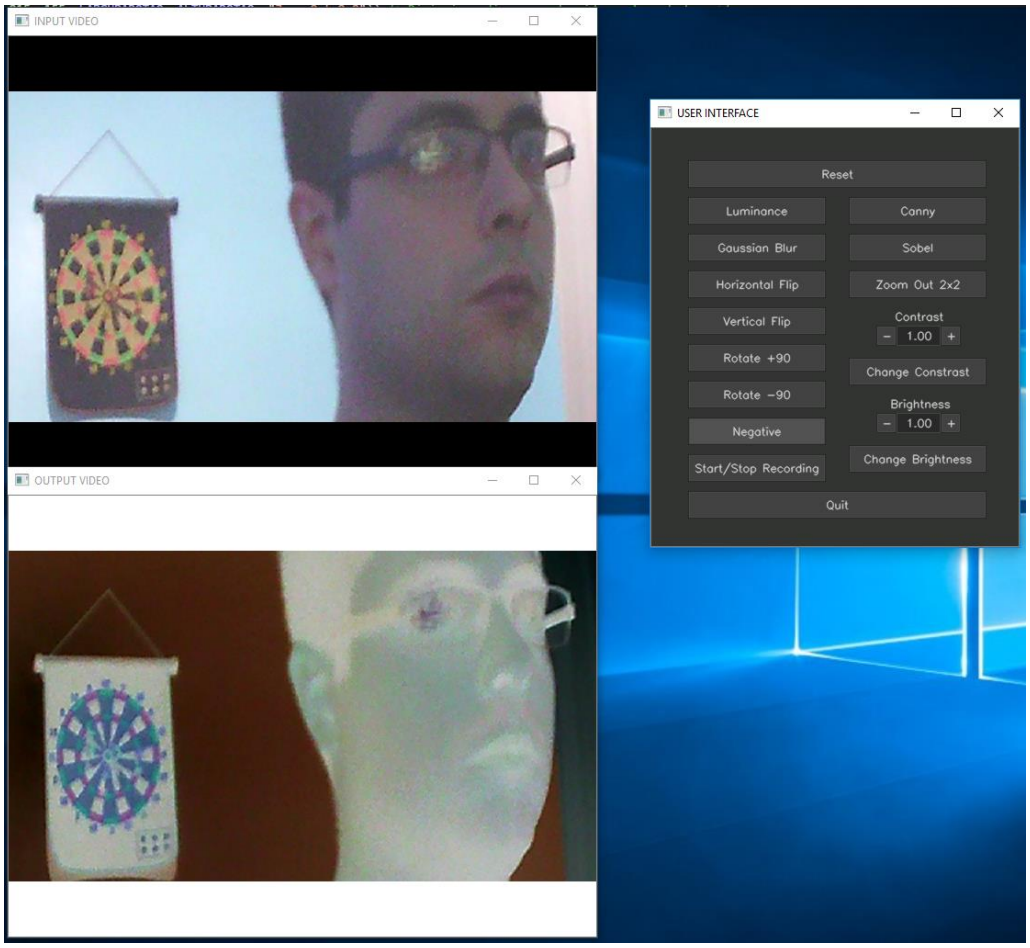


Abaixo, visualizamos os trechos do código que permitem essa implementação de mudança de brilho. Utilizou-se o comando **convertTo()** da biblioteca OpenCV.

```
if (cvui::button(frame_interface, 215, 345, LARGURABOTAO, ALTURABOTAO, "Change Brightness")) /* Altera o brilho do video. */
    comandos[OP_BRILHO] = 1;

if (comandos[OP_BRILHO] != 0) /* Verifica se o vetor de comandos contem a operacao de mudanca de brilho escolhida. */
    outCamera.convertTo(outCamera, -1, 1, brightness); /* Faz a mudanca de brilho. */
```

7) Obtenção do negativo do vídeo

Para obter o negativo dos frames capturados pela câmera, o usuário deve clicar em “Negative” na interface. A imagem abaixo demonstra essa operação realizada pelo programa.

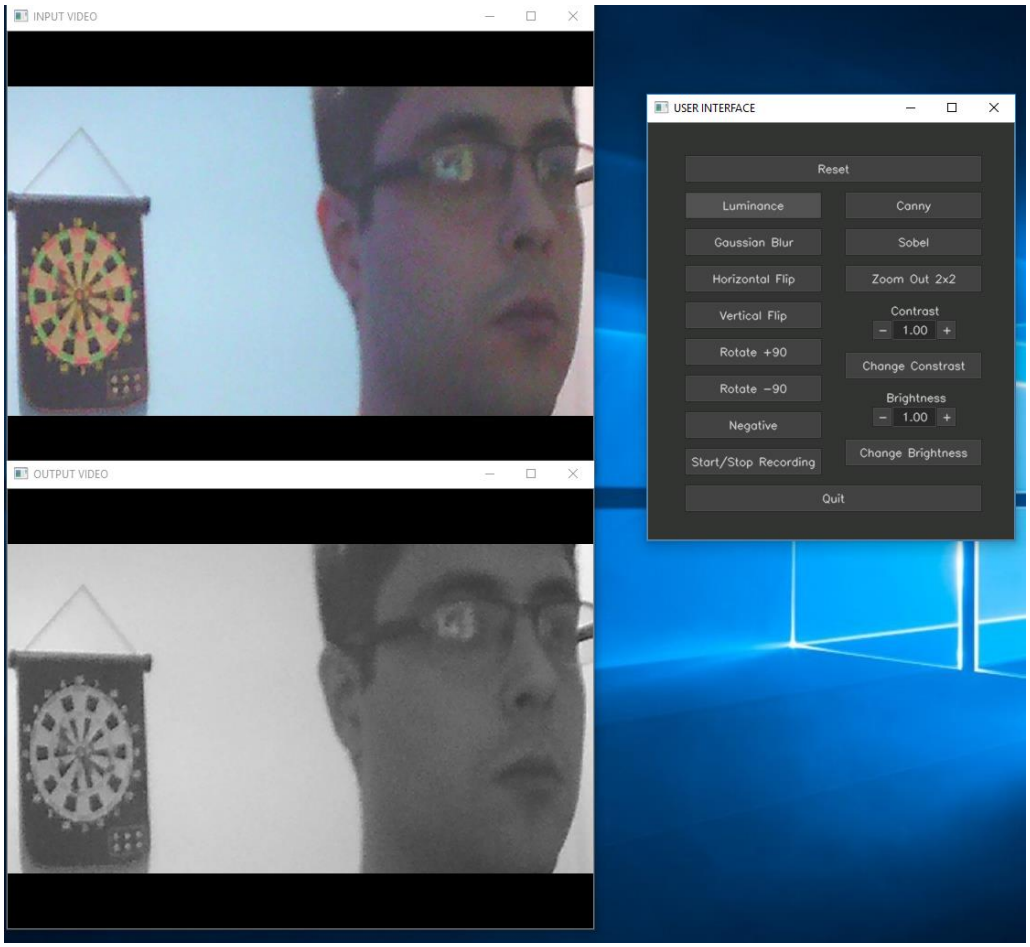


Abaixo, visualizamos os trechos do código que permitem essa implementação de cálculo de negativo. Utilizou-se o comando **convertTo()** da biblioteca OpenCV.

```
if (cvui::button(frame_interface, 40, 315, LARGURABOTAO, ALTURABOTAO, "Negative")) /* Calcula o negativo do video. */
{
    comandos[OP_NEGATIVO]++; /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */

    if (comandos[OP_NEGATIVO] == 2) /* Verifica se a operacao foi aplicada um numero par de vezes, para zerar o contador. */
        comandos[OP_NEGATIVO] = 0;
}

if (comandos[OP_NEGATIVO] != 0) /* Verifica se o vetor de comandos contem a operacao de negativo escolhida. */
    outCamera.convertTo(outCamera, -1, -1, 255);
```

8) Operação de luminância sobre o vídeo

Para aplicar a operação de luminância sobre os frames obtidos pela câmera, o usuário deve pressionar o botão “Luminance” na interface. A imagem abaixo demonstra essa operação realizada pelo programa.

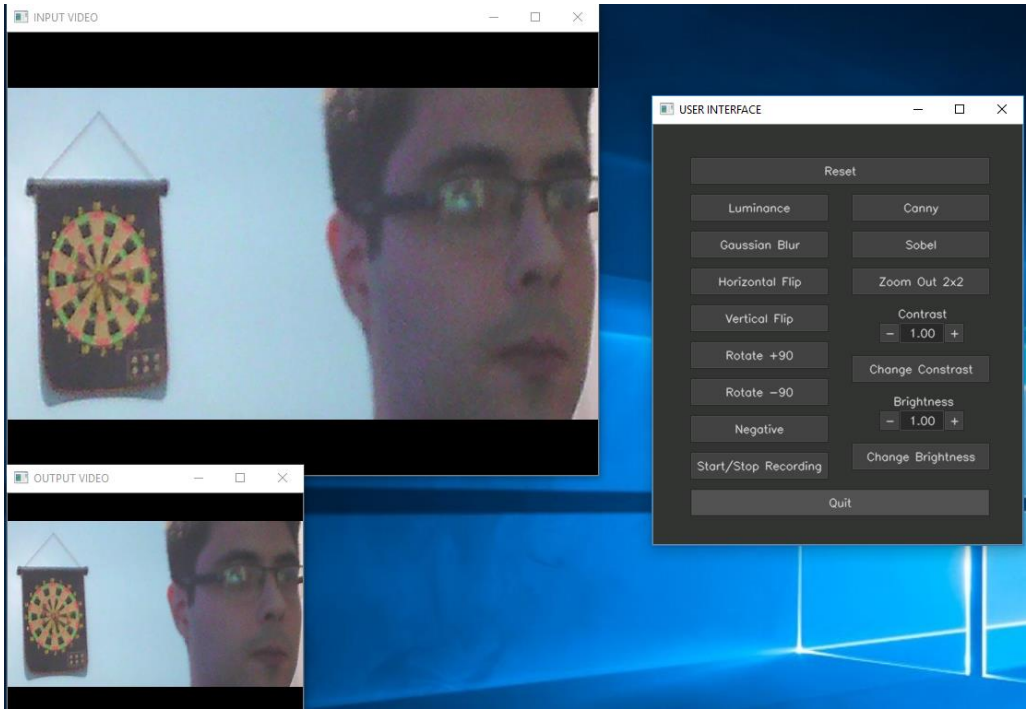


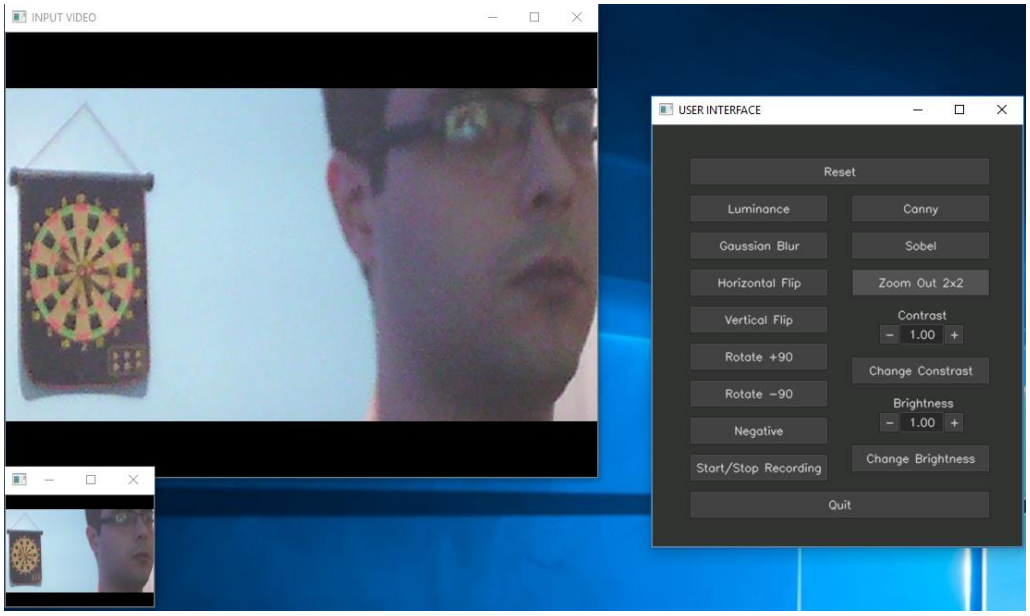
Abaixo, visualizamos os trechos do código que permitem essa implementação de operação de luminância. Utilizou-se o comando **cvtColor()** da biblioteca OpenCV.

```
if (cvui::button(frame_interface, 40, 75, LARGURABOTAO, ALTURABOTAO, "Luminance")) /* Aplica luminancia no video. */
    if (comandos[OP_LUMINANCIA] == 0) /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */
        comandos[OP_LUMINANCIA] = 1;

if (comandos[OP_LUMINANCIA] != 0) /* Verifica se o vetor de comandos contem a operacao de luminancia escolhida. */
{
    auxCamera = outCamera.clone();
    cvtColor(auxCamera, outCamera, COLOR_BGR2GRAY);
    cvtColor(outCamera, outCamera, COLOR_GRAY2BGR);
}
```

9) Redimensionamento do vídeo para metade do número de pixels em cada uma de suas dimensões

Para aplicar a operação de redimensionamento sobre os frames obtidos pela câmera, o usuário deve pressionar o botão “Zoom Out 2x2” na interface. As imagens abaixo demonstram duas operações sucessivas de redimensionamento, realizadas pelo programa.



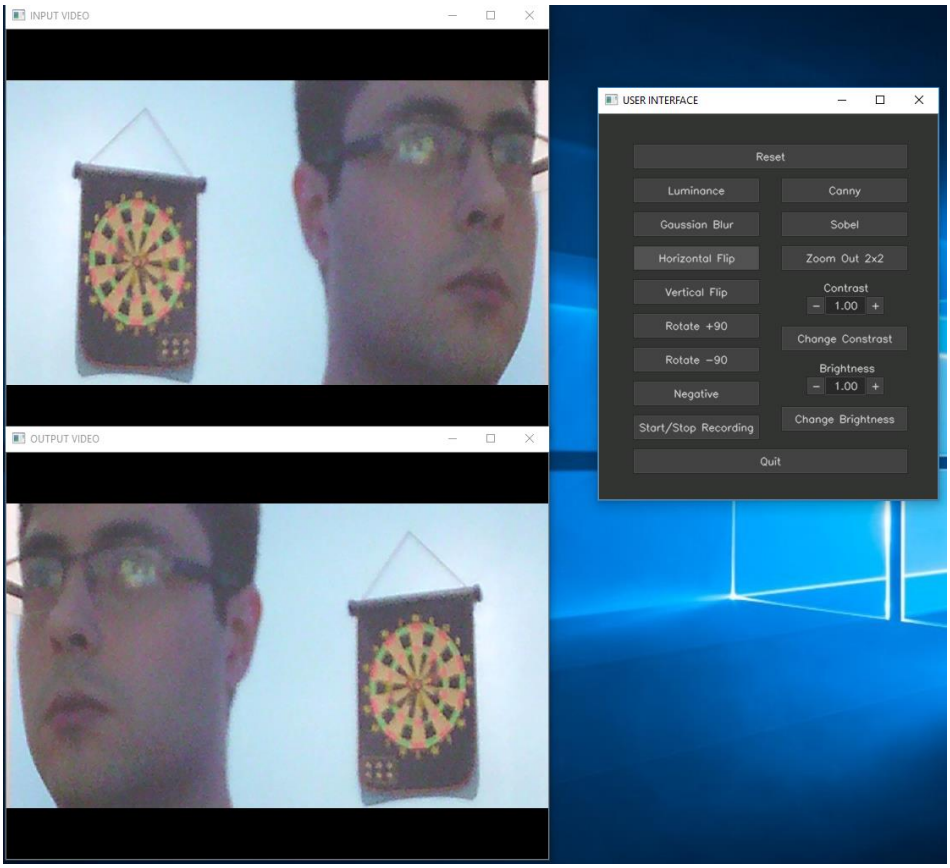


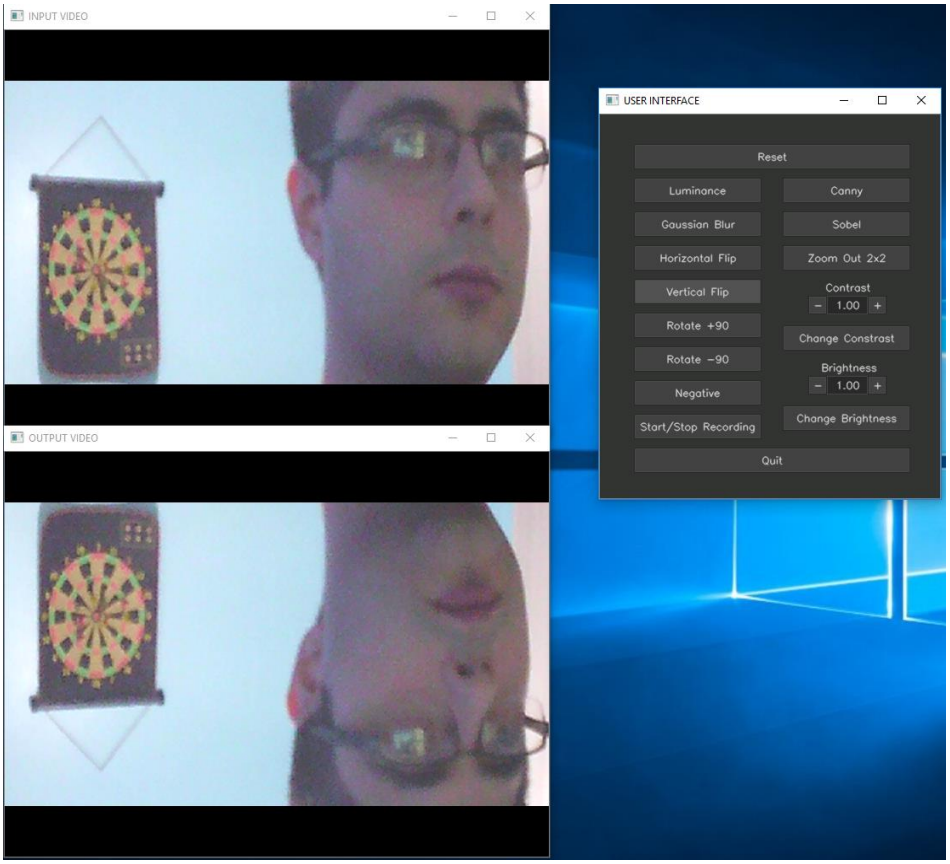
Abaixo, visualizamos os trechos do código que permitem essa implementação da operação de redimensionamento. Utilizou-se o comando **resize()** da biblioteca OpenCV.

```
if (cvui::button(frame_interface, 215, 155, LARGURABOTAO, ALTURABOTAO, "Zoom Out 2x2")) /* Diminui as dimensoes do video pela metade.  */
    comandos[OP_ZOOMOUT]++; /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */

if (comandos[OP_ZOOMOUT] != 0) /* Verifica se o vetor de comandos contem a operacao de Zoom Out escolhida. */
{
    for (int i = 0; i < comandos[OP_ZOOMOUT]; i++)
    {
        auxCamera = outCamera.clone();
        resize(auxCamera, outCamera, Size((int)auxCamera.cols / 2, (int)auxCamera.rows / 2));
    }
}
```

10) Espelhamentos horizontal e vertical sobre o vídeo

Para realizar os espelhamentos horizontal e vertical, o usuário deve pressionar os botões “Horizontal Flip” e “Vertical Flip” na interface. As imagens abaixo demonstram essas operações, realizadas pelo programa.





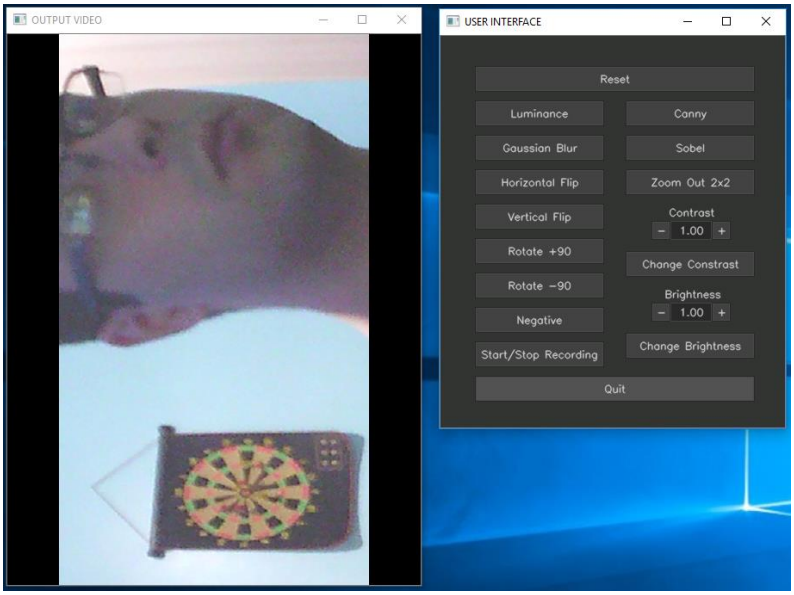
Abaixo, visualizamos os trechos do código que permitem essa implementação de operação de luminância. Utilizou-se o comando **flip()** da biblioteca OpenCV.

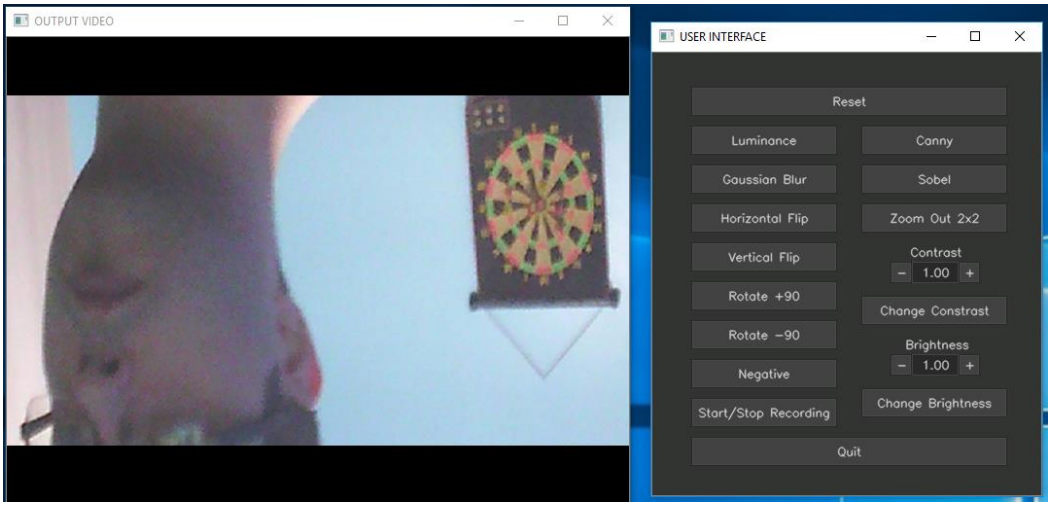
```
if (cvui::button(frame_interface, 40, 155, LARGURABOTAO, ALTURABOTAO, "Horizontal Flip")) /* Aplica espelhamento horizontal. */
{
    if (comandos[OP_FLIPHORIZONTAL] == 0)
        comandos[OP_FLIPHORIZONTAL]++; /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */

    else if (comandos[OP_FLIPHORIZONTAL] == 1)
        comandos[OP_FLIPHORIZONTAL] = 0;
}

if (comandos[OP_FLIPHORIZONTAL] == 1) /* Verifica se o vetor de comandos contem a operacao de espelhamento horizontal escolhida. */
    flip(outCamera, outCamera, CODIGOFLIPHORIZONTAL); /* Realiza o flip horizontal. */

if (cvui::button(frame_interface, 40, 195, LARGURABOTAO, ALTURABOTAO, "Vertical Flip")) /* Aplica espelhamento vertical. */
{
    if (comandos[OP_FLIPVERTICAL] == 0)
        comandos[OP_FLIPVERTICAL]++; /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */

    else if (comandos[OP_FLIPVERTICAL] == 1)
        comandos[OP_FLIPVERTICAL] = 0;
}

if (comandos[OP_FLIPVERTICAL] == 1) /* Verifica se o vetor de comandos contem a operacao de espelhamento vertical escolhida. */
    flip(outCamera, outCamera, CODIGOFLIPVERTICAL); /* Realiza o flip vertical. */
```

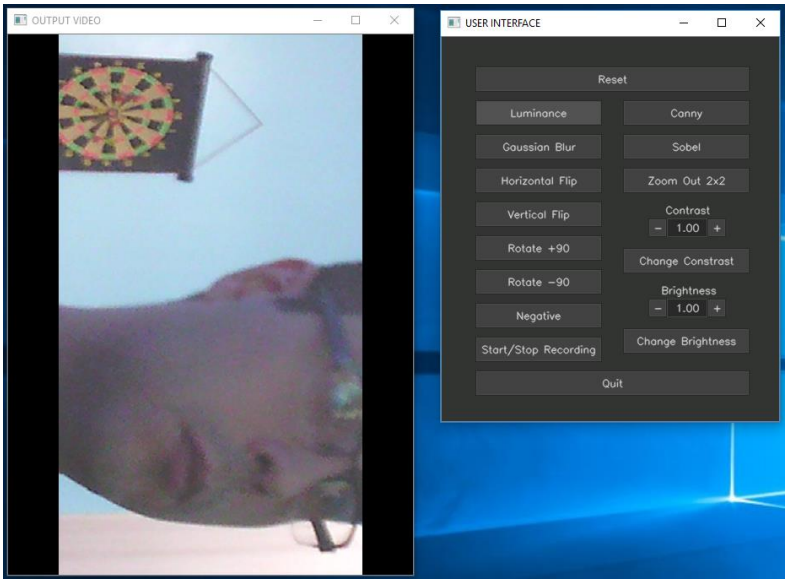
11) Rotações nos sentidos anti-horário e horário no vídeo

Para realizar as rotações nos sentidos anti-horário e horário no vídeo, o usuário deve utilizar os botões “Rotate +90” e “Rotate -90” na interface. As imagens abaixo demonstram essas operações, realizadas pelo programa. A janela contendo o vídeo inalterado foi omitida, para facilitar a visualização.

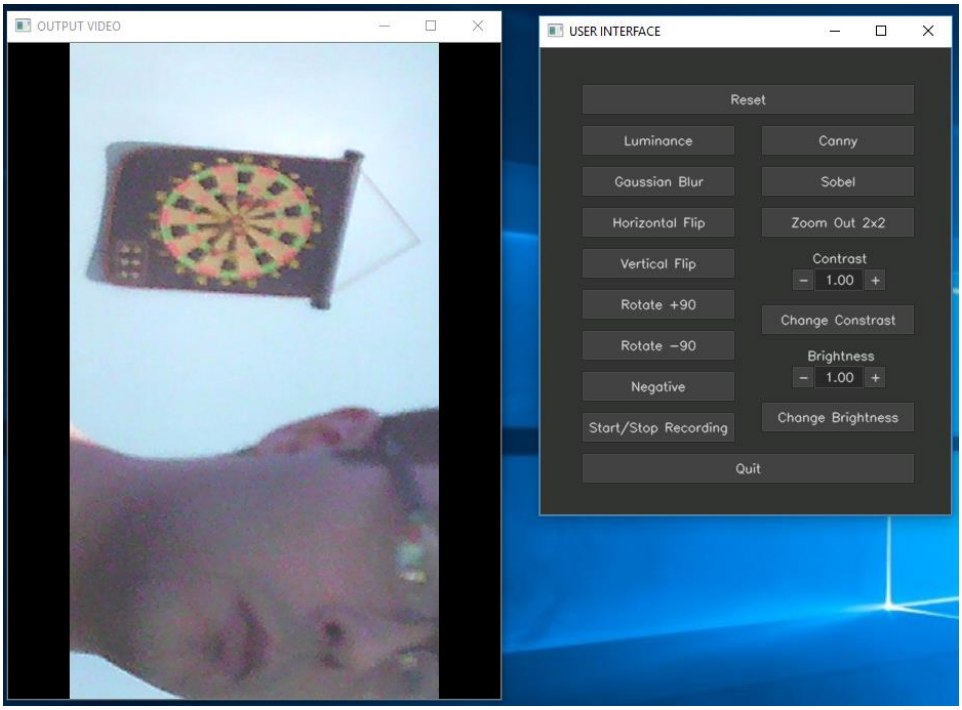
Rotações sucessivas no sentido anti-horário:

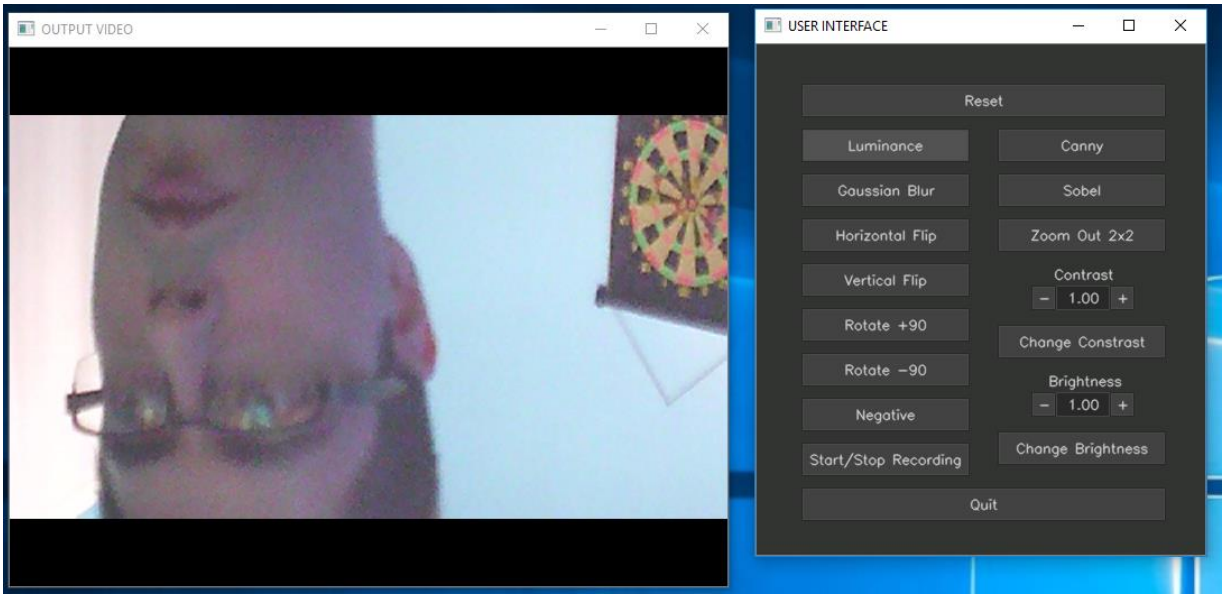


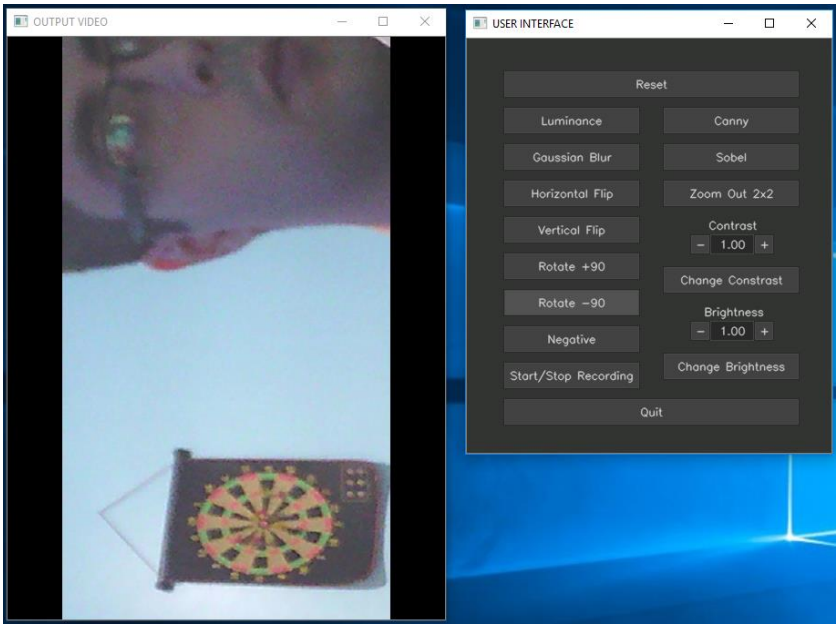




Rotações sucessivas no sentido horário:







Abaixo, visualizamos os trechos do código que permitem essas rotações. Utilizou-se o comando **rotate()** da biblioteca OpenCV.

```
if (cvui::button(frame_interface, 40, 235, LARGURABOTAO, ALTURABOTAO, "Rotate +90"))/* Rotaciona o video em +90 graus. */
{
    comandos[OP_ROTACIONAHORARIO]++; /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */

    if (comandos[OP_ROTACIONAHORARIO] == 4)
        comandos[OP_ROTACIONAHORARIO] = 0;
}

if (comandos[OP_ROTACIONAHORARIO] != 0) /* Verifica se o vetor de comandos contem a operacao de rotacao em +90 graus escolhida. */
    for (int i = 0; i < comandos[OP_ROTACIONAHORARIO]; i++)
        cv::rotate(outCamera, outCamera, cv::ROTATE_90_COUNTERCLOCKWISE);

if (cvui::button(frame_interface, 40, 275, LARGURABOTAO, ALTURABOTAO, "Rotate -90")) /* Rotaciona o video em -90 graus. */
{
    comandos[OP_ROTACIONAANTIHORARIO]++; /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */

    if (comandos[OP_ROTACIONAANTIHORARIO] == 4)
        comandos[OP_ROTACIONAANTIHORARIO] = 0;
}

if (comandos[OP_ROTACIONAANTIHORARIO] != 0)  /* Verifica se o vetor de comandos contem a operacao de rotacao em +90 graus escolhida. */
    for (int i = 0; i < comandos[OP_ROTACIONAANTIHORARIO]; i++)
        cv::rotate(outCamera, outCamera, cv::ROTATE_90_CLOCKWISE);
```

12) Gravação de vídeo

Para iniciar a gravação de um vídeo, o usuário deve pressionar o botão “Start/Stop Recording”. Uma vez pressionado esse botão, a gravação é iniciada, e ela é interrompida no momento em que o programa é encerrado utilizando o botão “Quit” da interface, ou pressionando a tecla ESC. Quando a gravação é interrompida, é gerado o arquivo de saída “videosaida.avi”, no mesmo diretório em que se encontra o arquivo executável (.exe) do programa.

Um exemplo de vídeo gerado pelo programa pode ser visualizado no link a seguir. Nele, são demonstradas todas as operações do programa (exceto rotação e dimensionamento), em tempo real.

https://streamable.com/7l35vb

Abaixo, visualizamos os trechos de código que permitem a gravação do vídeo.

```
int frame_largura = cap.get(cv::CAP_PROP_FRAME_WIDTH);
int frame_altura = cap.get(cv::CAP_PROP_FRAME_HEIGHT);
VideoWriter video(NOMEVIDEOSAIDA, cv::VideoWriter::fourcc('M', 'J', 'P', 'G'), 10, Size(frame_largura, frame_altura));
```

```
if (cvui::button(frame_interface, 40, 355, LARGURABOTAO, ALTURABOTAO, "Start/Stop Recording")) /* Cria um arquivo de video com alguns frames. */
{
    if (comandos[OP_VIDEO] == 0) /* Atualiza o vetor de comandos, caso o comando tenha sido escolhido. */
        comandos[OP_VIDEO] = 1;

    else if (comandos[OP_VIDEO] == 1) /* Encerra a gravacao do video, caso o usuario pressione a tecla novamente.  */
    {
        comandos[OP_VIDEO] = 0;
        video.release();
    }
}
```

DIFICULDADES ENFRENTADAS

Nesse trabalho, não houve grandes dificuldades enfrentadas, e todas as operações puderam ser implementadas.

REFERÊNCIAS

https://www.youtube.com/watch?v=2FYm3GOonhk&list=LL&index=9

https://docs.opencv.org/3.4/d4/d13/tutorial_py_filtering.html

https://docs.opencv.org/3.4/da/d6a/tutorial_trackbar.html

https://docs.opencv.org/3.4/da/d22/tutorial_py_canny.html

https://docs.opencv.org/3.4/d2/d2c/tutorial_sobel_derivatives.html

https://docs.opencv.org/3.4/de/d25/imgproc_color_conversions.html

https://www.tutorialspoint.com/how-to-rotate-a-video-in-opencv-using-cplusplus

https://www.educba.com/opencv-flip/

https://learnopencv.com/read-write-and-display-a-video-using-opencv-cpp-python/